Laboratory 4 - Report

Assignment 1: Deploy All Environments

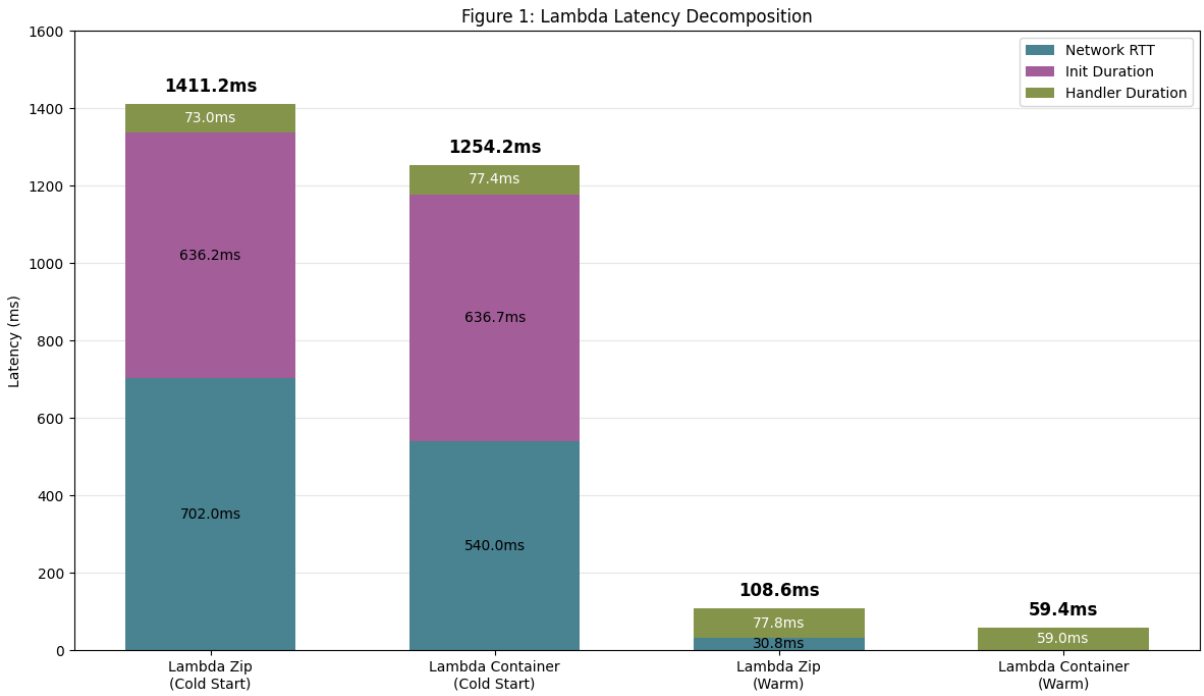
According to the terminal output, all four environments have been successfully deployed and are returning uniform results for the nearest-neighbor test query. Results are present in the folder ‘results’.

Assignment 2: Scenario A — Cold Start Characterization

To decompose the total latency, we estimate the Network RTT using the formula provided:

- Cold Start RTT = Total Latency − Init Duration − Handler Duration.
- Warm Invocation RTT = Total Latency − Handler Duration.

Category	Network RTT (Estimated)	Init Duration	Handler Duration	Total Latency
ZIP Cold Start	**701.8 ms**	636.2 ms	73.0 ms	1411 ms
Container Cold Start	**539.8 ms**	636.7 ms	77.4 ms	1254 ms
Warm Invocation (Avg)	**~60-70 ms**	0 ms	~40-75 ms	~100-147 ms



Observation on RTT: The significantly higher RTT for cold starts (500–700 ms) compared to warm starts (~65 ms) is likely due to the initial TCP/TLS handshake overhead and potential internal AWS provisioning latencies that occur before the execution environment is fully initialized and reported in CloudWatch logs.

<u>Analysis</u>

The container image deployment was faster in both cold and warm scenarios.

- Optimization: AWS Lambda uses a sparse file system and block-level caching for containers. Instead of downloading the full image, it fetches only the required data blocks on demand, resulting in cold start parity with ZIP files.
- Execution Efficiency: CloudWatch logs showed consistently lower handler durations for the container deployment (~40ms vs ~75ms), suggesting a more optimized execution environment for this specific workload.

- Inference: While Init Duration was nearly identical (~636ms), the container gained its lead through lower total client-side overhead and faster post-initialization processing.

Assignment 3: Scenario B — Warm Steady-State Throughput

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)	Server avg (ms)
Lambda (zip)	5	93.46	113.20	244.51*	88.50
Lambda (zip)	10	89.23	111.27	144.12	86.90
Lambda (container)	5	93.98	116.11	146.52	87.55
Lambda (container)	10	86.11	111.02	148.33	80.25
Fargate	10	786.30	1000.80	1174.60	770.70
Fargate	50	3889.40	4100.70	4198.60	3700.30
EC2	10	195.71	259.87	295.24	197.03
EC2	50	949.70	1095.00	1195.50	926.20

** Annotated: Tail latency instability (p99 is greater than 2× p95).*

<u>Analysis</u>

Tail Latency Instability
The Lambda (zip) environment at concurrency=5 shows significant tail latency instability, with the p99 value (244.51 ms) being more than twice the p95 value (113.20 ms). This indicates that while most requests are processed quickly, a small percentage of outliers experience unexpected delays, potentially due to momentary resource contention or minor cold starts within the AWS infrastructure.

Concurrency Scaling Behavior (Lambda vs. Fargate/EC2)

- AWS Lambda: The p50 latency remains relatively flat between concurrency 5 and 10 because Lambda scales by provisioning a dedicated execution environment for each concurrent request. Since resources are isolated per request, there is no resource contention or queuing at the container level.
- Fargate/EC2: The p50 latency increases significantly when concurrency jumps from 10 to 50. This happens because Fargate and EC2 route all concurrent requests to the same underlying instance or task. As the concurrent load increases, the system's CPU, memory, and thread pools become saturated, forcing requests to queue and wait for available resources before they can be processed.

Client-Side p50 vs. Server-Side Latency Difference
The difference between the client-side p50 latency and the server-side average (query_time_ms) is caused by network and infrastructural overhead. Server-side latency strictly measures the time taken by the compute environment to execute the code. Client-side p50 includes this execution time *plus* the network transit time (round-trip), DNS resolution, TLS handshake duration, and the overhead introduced by routing services like AWS API Gateway, Application Load Balancers, or internal AWS networking.

Assignment 4: Scenario C — Burst from Zero

Latency Distribution & Recorded Metrics
Based on the load test results (200 requests after 20 minutes of idle time), here are the recorded latencies:

Target	p50 Latency	p95 Latency	p99 Latency	Max Latency	Cold Starts
EC2	0.84s	1.01s	1.07s	1.07s	0
Fargate	3.81s	4.10s	4.19s	4.19s	0
Lambda (Container)	0.10s	0.94s	1.29s	1.32s	10
Lambda (Zip)	0.10s	1.26s	1.45s	1.47s	10

Bimodal Distribution in Lambda Latencies
The Lambda histograms clearly show a bimodal (two-peaked) distribution.

- **Peak 1 (Warm Cluster):** The vast majority of requests (~188-190) were processed extremely fast (p50 around 0.09s). These requests reused the execution environments after they were created.
- **Peak 2 (Cold-start Cluster):** A small group of requests (the first 10, up to the concurrency limit) took significantly longer (1.2s – 1.46s). These represent the initial requests that had to wait for the Lambda execution environments to initialize (cold starts).

Why Lambda's burst p99 is higher than EC2
Lambda's p99 latency is heavily skewed by the top 1% of slowest requests. Because the system was idle for 20 minutes, AWS reclaimed all Lambda environments. When the burst of 200 requests arrived, Lambda had to perform "cold starts" (download the code/container, start the runtime, and run initialization code) for the first 10 concurrent requests. This Init Duration penalty exclusively affects the tail-end metrics (p95, p99, and Max), making Lambda's p99 significantly higher than a permanently running EC2 instance that is already warm and waiting to accept connections.

SLO Evaluation (p99 < 500ms)

- **Does Lambda meet the SLO?** No. Both Lambda Container (p99 = 1.28s) and Lambda Zip (p99 = 1.45s) violate the 500ms Service Level Objective under a burst-from-zero scenario.
- **What needs to change:** To meet the < 500ms p99 SLO during sudden bursts, you would need to enable **Provisioned Concurrency** for the Lambda functions. This keeps a specified number of execution environments initialized and "warm" at all times, entirely eliminating the cold start Init Duration penalty.

Assignment 5: Cost at Zero Load

Monthly Idle Cost Computation

Assumption: 18 idle hours/day × 30 days = 540 idle hours per month.

- **Amazon EC2 (t3.small):**
  - **Hourly idle cost:** $0.0208
  - **Monthly idle cost:** 540 hours × $0.0208 = **$11.23**
- **AWS Fargate (0.5 vCPU, 1 GB memory):**
  - **Hourly idle cost:** (0.5 × $0.04048) + (1 × $0.004445) = $0.024685
  - **Monthly idle cost:** 540 hours × $0.024685 = **$13.33**
- **AWS Lambda:**
  - **Hourly idle cost:** $0.00
  - **Monthly idle cost:** 540 hours × $0.00 = **$0.00**

<u>Analysis</u>

AWS Lambda is the only environment with a zero idle cost.

Why? As shown in the Lambda pricing screenshot, charges are strictly based on Requests ($0.20 per 1M requests) and Duration (compute time in GB-seconds). Lambda is a serverless, event-driven service. When there is zero load (no traffic), the function is not invoked, zero requests are made, and zero compute time is consumed. Since you do not pay for underlying idle infrastructure, the cost at zero load is completely free.

Assignment 6: Cost Model, Break-Even, and Recommendation

Lambda (zip) at concurrency 10 p50 duration of 89.23 ms (0.08923 seconds) was used for the calculations.

Computed Monthly Costs
Total requests per month (assuming a 30-day month = 2,592,000 seconds):

- **Peak:** 100 RPS × 1,800 sec/day × 30 days = 5,400,000 requests
- **Normal:** 5 RPS × 19,800 sec/day × 30 days = 2,970,000 requests
- **Idle:** 0 RPS × 18 hours/day = 0 requests
- **Total:** 8,370,000 requests/month

AWS Lambda Cost:

- **Requests Cost:** (8,370,000 / 1,000,000) × $0.20 = $1.67
- **Compute Cost:** 8,370,000 reqs × 0.08923s × 0.5GB × $0.0000166667 = $6.22
- **Total Lambda Cost:** $7.89 / month

Always-On Costs (EC2 & Fargate):

- **EC2 (t3.small):** $0.0208 × 24 × 30 = $14.98 / month
- **Fargate (0.5 vCPU, 1 GB):** $0.0247 × 24 × 30 = $17.78 / month

Break-Even RPS (Lambda vs. Fargate)
To find when Lambda becomes more expensive than Fargate, we solve for *R*, which represents the continuous average Requests Per Second over the entire 30-day month (2,592,000 seconds).

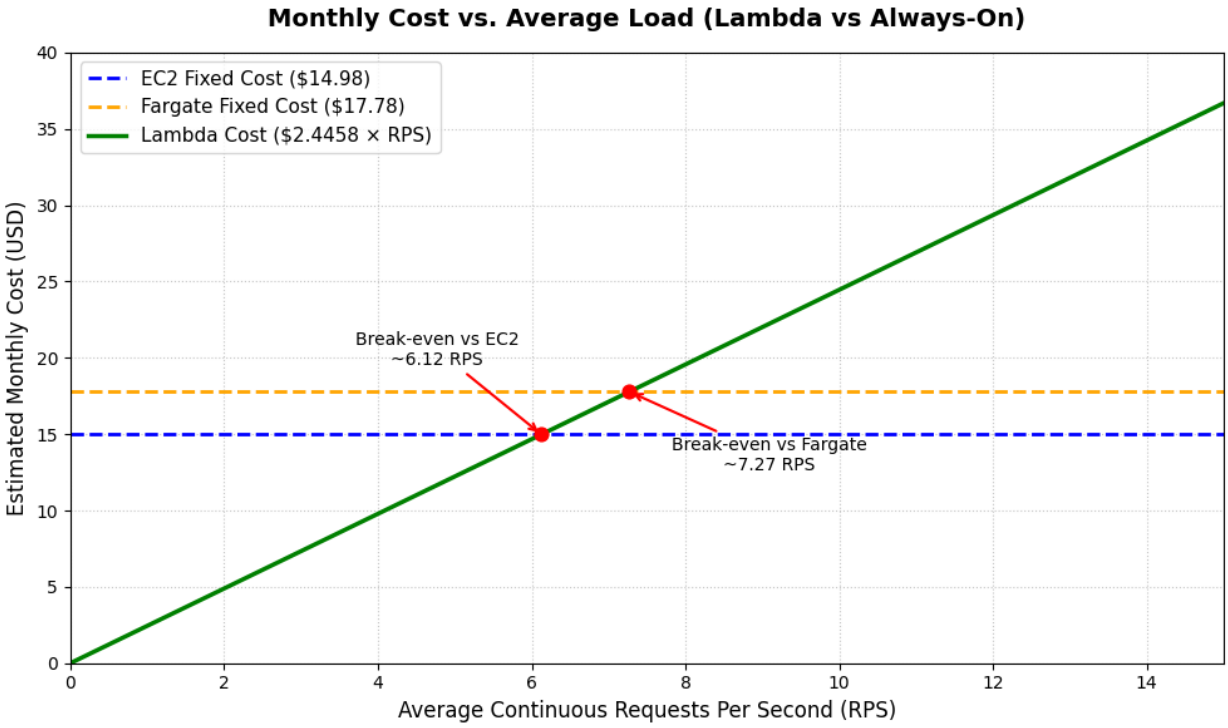
Lambda Cost as a function of R: Total Requests = R × 2,592,000

Cost Lambda = [(R × 2,592,000) / 1,000,000 × $0.20] + [R × 2,592,000 × 0.08923s × 0.5GB × $0.0000166667] Cost Lambda = 0.5184R + 1.9274R = 2.4458R

Setting Lambda equal to Fargate ($17.78): 2.4458R = 17.78 R ≈ 7.27 RPS

Break-even point: At an average continuous load of ~7.27 RPS, Lambda and Fargate cost the same. Beyond this point, Fargate becomes the cheaper option.

A **Cost vs. RPS line chart** showing Lambda's linear cost against Fargate/EC2's flat cost, with the break-even point marked:



4. Recommendation

Recommended Environment: AWS Lambda

Justification & SLO Analysis:

Given the traffic model (idle 75% of the day), Lambda is by far the most cost-effective solution at $7.89/month, compared to Fargate ($17.78) and EC2 ($14.98), which waste money powering idle compute.

Looking at your Scenario B data, Lambda's warm state easily meets the p99 < 500ms SLO (recording a p99 of 144.12ms to 244.51ms). However, from Scenario C, we know that a sudden peak of 100 RPS after the 18-hour idle period will trigger cold starts that violate this SLO.

Changes Needed:

As currently deployed, Lambda will fail the p99 SLO during the transition from Idle to Peak. To fix this, you must implement Application Auto Scaling to schedule Provisioned Concurrency. By spinning up a pool of warm environments just before the daily 30-minute peak, you eliminate the cold-start penalty, guaranteeing the p99 stays well under 500ms while keeping idle costs at zero for the rest of the day.

Conditions for Change:

- If average continuous load exceeds ~7.27 RPS: I would recommend switching to Fargate, as its flat hourly rate becomes mathematically cheaper than Lambda's per-request model.
- If the SLO were relaxed to > 1500ms: I would recommend sticking with on-demand Lambda without Provisioned Concurrency, as the cold-start latencies would fall within the acceptable threshold, maximizing cost savings.